MODULE 5

User Authentication & Sessions

Login, Sessions, Flash Messaging, and RBAC in Flask.

Core Topics

- Flask-Login
- Sessions & Cookies
- Flash Messaging

Implementation

- Registration Forms
- Profile Updates
- Role-Based Access

PART 1

Flask-Login

Extension to manage authentication and user sessions.

- Logs users in and out.
- Remembers authenticated state.
- Protects specific routes.

```
# Installation
pip install flask-login

# Basic Setup
from flask_login import LoginManager

login_manager = LoginManager()
login_manager.init_app(app)

# Redirect unauthenticated users
login_manager.login_view = 'login'
```

Login & Logout

Login Flow

1. Enter credentials.
2. Find user in DB.
3. Verify credentials.
4. Call **login_user(user)**.

```
from flask_login import login_user
# ... verification ...
login_user(user)
```

Logout Route

Clears the user's session.

```
from flask_login import logout_user

@app.route('/logout')
def logout():
    logout_user()
    return redirect('/login')
```

Protecting Pages

The **@login_required** decorator blocks unauthenticated guests.

Guest User

🚫 Redirected to Login Page.

Logged-in User

✓ Access Granted.

```python
from flask_login import login_required, current_user

@app.route('/dashboard')
@login_required
def dashboard():
    return "Welcome to Dashboard"

@app.route('/profile')
@login_required
def profile():
    # Access logged in user
    return current_user.username
```

PART 2

Sessions in Flask

Maintains state across stateless HTTP requests.

- Requires a**secret_key**.
- Used to sign session data securely.

```python
from flask import session

# App config
app.secret_key = 'my-secret-key'

# Write to session
session['username'] = 'Nilesh'

# Retrieve
user = session.get('username')

# Remove
session.pop('username', None)
```

Cookies & State Management

Session	Cookie
Maintains user state.	Stores small data in browser.
Flask provides built-in support.	Managed through the browser.
Flask's default is cookie-based & signed.	Sent with HTTP requests.
Useful for temporary app data.	Useful for preferences.

PART 3

Flash Messaging

Temporary messages displayed after actions (e.g., login, updates).

```python
from flask import flash

# Basic message
flash("Login successful!")

# With a category
flash("Invalid password!", "error")
```

success error warning info

Lifecycle

1. User performs action
↓
2. flash() creates message
↓
3. Redirect / Next request
↓
4. Message displayed (once)

Displaying Messages in HTML

Rendered via Jinja templating engine.

Basic Loop

```
{% with messages = get_flashed_messages() %}
  {% for message in messages %}
    <p>{{ message }}</p>
  {% endfor %}
{% endwith %}
```

With Categories

```
{% with messages =
  get_flashed_messages(with_categories=true) %}

  {% for category, message in messages %}
    <p class="{{ category }}">
      {{ message }}
    </p>
  {% endfor %}

{% endwith %}
```

PART 4

Registration Flow

Form Data → Validate → Check Exists → Save DB → Flash Success

Crucial Validations

- ⚠️ Required fields not empty.
- ⚠️ Passwords must match.
- ⚠️ Username/Email must be unique.
- 🔒 **Hash passwords** securely.

```
# Controller logic
user = User.query.filter_by(
    username=username
).first()

if user:
    flash("Username exists!", "error")
    return redirect('/register')
```

PART 5

Profile Updates

Requires **@login_required** to ensure only the authenticated user modifies their data.

HTML Template Snippet:

```
<input type="text" name="username"
value="{{ current_user.username }}">
```

```python
@app.route('/profile', methods=['GET', 'POST'])
@login_required
def profile():
    if request.method == 'POST':

        current_user.username = \
            request.form['username']

        current_user.email = \
            request.form['email']

        db.session.commit()
        flash("Profile updated!", "success")
        return redirect('/profile')

    return render_template('profile.html')
```

PART 6

Role-Based Access Control

Controls access to resources based on assigned roles, rather than per-user permissions.

Role	Access Level
Admin	Manage app & users
Faculty	Manage academic info
Student	View academic info

Role Checking Logic

```
@app.route('/admin')
@login_required
def admin():

    # Verify authorization
    if current_user.role ≠ 'admin':
        return "Access Denied"

    return "Admin Dashboard"
```



Module Summary

Flask-Login: Handle auth state securely.

Sessions: Maintain server-side user data.

Cookies: Browser-side storage.

Flash Messages: Temporary UI feedback.

Registration/Profile: Secure forms & updates.

RBAC: Authorize via roles, not just identity.